also assess whether event handlers trigger the correct responses, such as clicks or form submissions, and produce the desired outcomes.

INTEGRATION TESTING

Integration testing examines how different parts of the application work together. This includes testing interactions between components, API routes, and external services such as generative AI models. For example, when an API route is called to generate a response from an LLM, integration tests can verify that the application correctly sends requests and handles responses. Developers should focus on API endpoints and handlers, how the data flows between boundary systems, and component integration:

- *API endpoints*—Integration tests evaluate how well the application interacts with external APIs. This includes testing whether API calls return the expected data and whether the application can handle various response scenarios effectively.
- *Data flow*—These tests ensure that data flows correctly between components and services within the application. They validate that data passed from one component to another is accurate and that any transformations applied to the data are performed correctly.
- *Component integration*—Integration testing checks how different components work together as a unit as they perform interactions with each other.

Overall, this type of testing is more expensive to run, both in terms of infrastructure and running speed. By employing both unit and integration testing strategies, developers can achieve a robust testing framework that ensures a stable delivery of new features.

8.4.2 Mocking LLM responses

Mocking is a testing technique where you replace real system dependencies with simulated objects that mimic the behavior of the real components. These controlled environments allow you to test various scenarios without the added costs of real model interactions. In LLM testing, mocking involves creating artificial responses that simulate what an LLM would return, without calling the LLM API.

Mocking LLM responses is useful for several practical reasons. Real LLM API calls are expensive and slow, making frequent testing costly and time consuming. LLMs are nondeterministic and return different responses to identical prompts, which makes writing reliable tests more difficult. Mocking allows you to isolate your code logic from external dependencies so that tests focus on your application rather than the LLM service's behavior.

CHALLENGES AND CONCERNS OF MOCKING

Mocking LLM responses presents several significant challenges. The authenticity of the mocked responses is particularly obvious when dealing with context-dependent tasks such as a response to a carefully crafted prompt. A mock might return a simple predefined response for a sentiment analysis task, while a real LLM would consider more intricate factors to determine the output.